



## 1 Contexte et objectifs

Application nommée **LocDVD** de sélection de films classés par catégorie.  
Nous allons afficher des données enregistrées dans un tableau.

### 1.1 Présentation

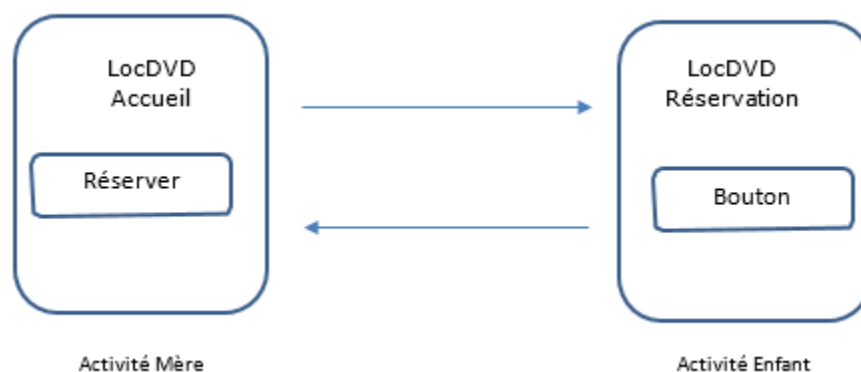
Ce guide partiel constitue le support privilégié pour la création d'applications Android multi-activités.

Jusqu'à présent, nous n'avons créé que des applications mono-activité. En général les applications comportent plusieurs écrans gérés par des activités. C'est ce que nous allons voir dans ce TP n°5.

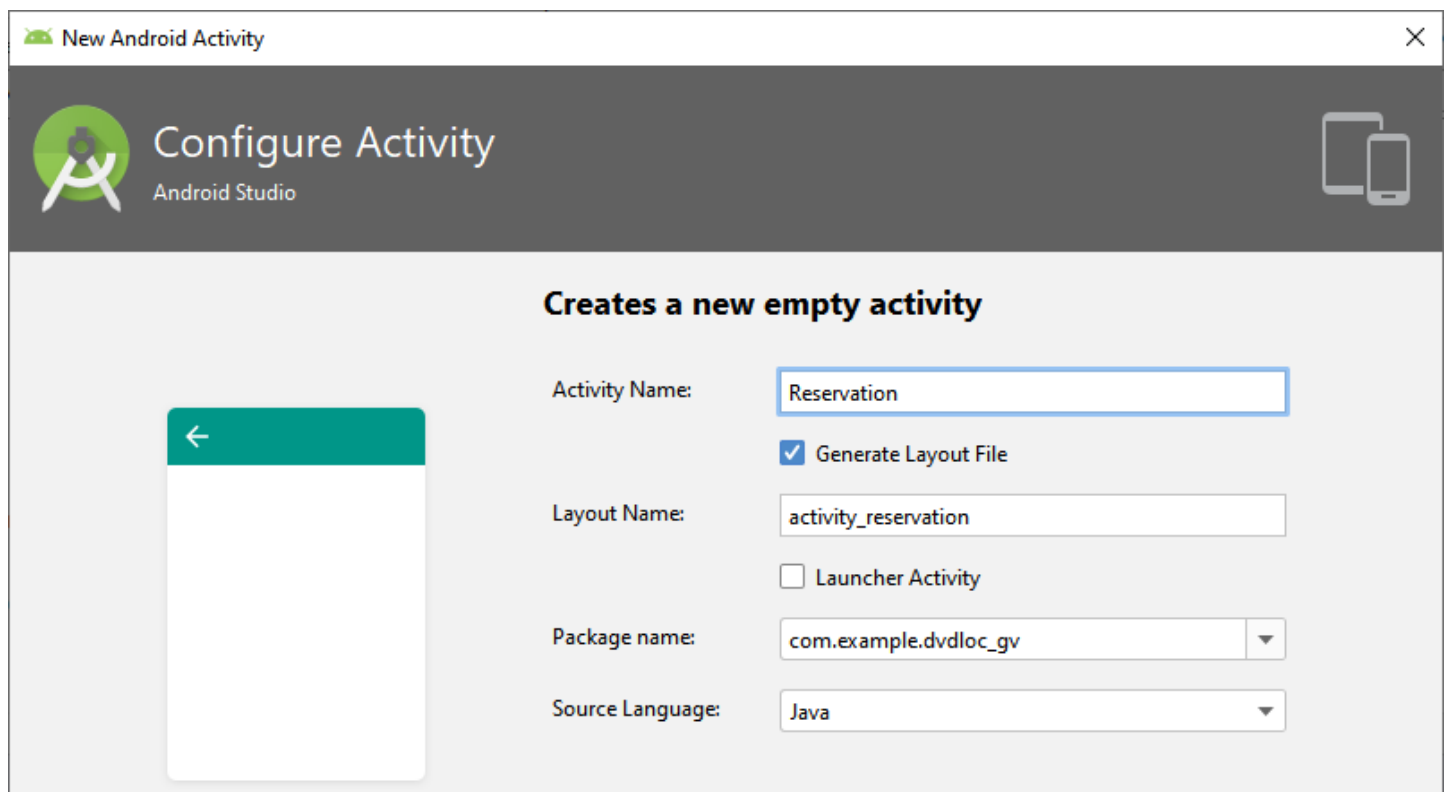
## 2 Activité Réservation : Communiquer un retour

### 2.1 Principe

Nous allons créer une activité qui va retourner un résultat. Selon le résultat reçu, nous afficherons tel ou tel résultat sur l'activité principale.



### 2.2 Création d'une nouvelle l'activité de Reservation



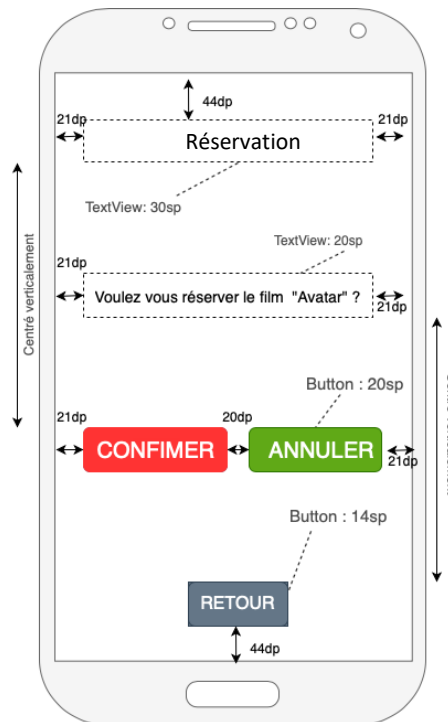


A. Créez l'**Activité Réservation**, en partant d'une Activité de type empty :

- **File / New / Activity / EmptyActivity**
- La création de l'activity : **Reservation** génère le layout : **activity\_reservation**

B. Compléter le **layout** comme indiqué ci-dessous sur l'écran, sachant que :

- Les textes sont centrés.
- Tous les composants auront un **id** significatif et unique, que vous aurez choisi.
- Créez les ressources **string**, qui correspondent.
- Modifiez la couleur des boutons.



## 2.3 Liaison des activités entre elles

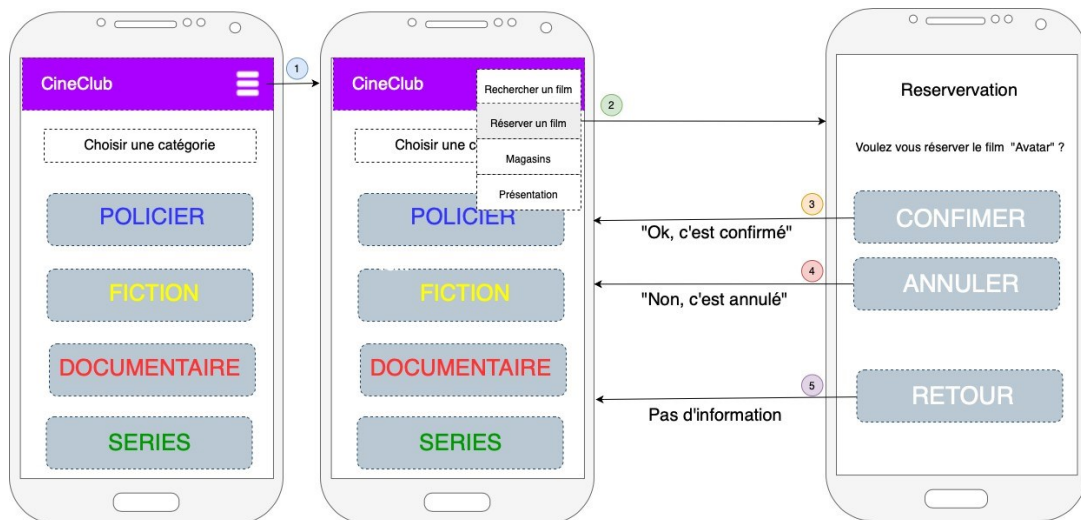
Appeler le layout **Reservation** au clic sur le menu **Réserver un film** (dans **MainActivity.java**)

```
switch (item.getItemId())
{
    case R.id.menuRechercher:
        //Log.i("LocDVD", "Menu:Rechercher un film");
        return true;

    case R.id.menuReserver:
        //Log.i("LocDVD", "Menu:Réserver un film");
        //Lance l'activité qui affiche la fenêtre Réservation
        Intent intent = new Intent(MainActivity.this, Reservation.class);
        startActivity(intent);
        return true;
}
```



Le bouton « **RETOUR** » doit permettre de revenir vers l'Activité principale.



## 2.4 Exemple de passage de données entre activités

### 1.1.1 Retour à l'activité Principale au clic sur Confirmer

Pour cela vous devez modifier les classes java des activités.

#### Réservation.java

On ajoute un attribut privé, **btnConfirmer** de type **Button**. Comme pour le bouton retour, on lui associe un écouteur d'événement.

```
public class Reservation extends AppCompatActivity {
    private Button btnConfirmer;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_reservation);

        // Confirmation
        btnConfirmer = findViewById(R.id.btnConfirmerResa);
        btnConfirmer.setOnClickListener(new Button.OnClickListener()
        {
            @Override
            public void onClick(View view) {
                setResult(Activity.RESULT_OK);
                finish();
            }
        });
    }
}
```

- **Activity.RESULT\_OK** est une constante statique de la classe Activity. Elle est utilisée comme un code -1 = OK
- **finish()** est une méthode qui permet de finir l'activité. On repasse alors sur l'activité « appelante », MainActivity pour nous.



### 1.1.2 Retour de données d'une activité enfant

#### Principes

Pour l'instant, nous savons démarrer une activité à partir d'une autre. Mais il n'y a aucune communication entre les activités, puisqu'elles ne s'échangent aucune donnée.

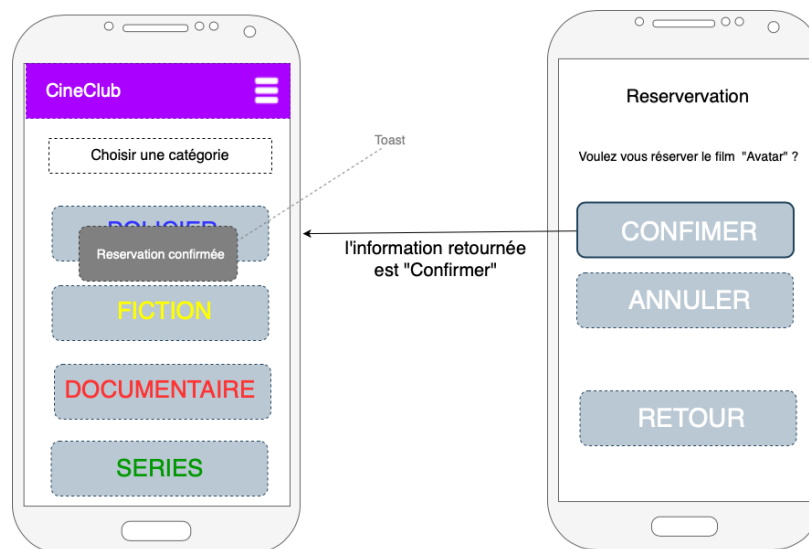
Une vraie communication consisterait à permettre au moins à l'activité **Réservation** de retourner un résultat à l'activité **MainActivity**.

Nous avons vu que la méthode **startActivity(intent)** est incapable d'assurer cette mission.

Par contre l' API **ActivityResultLauncher** : permet de gérer un retour :

- Elle est appelée avec la méthode **launch** et prend en paramètre l'intent

Au click du bouton confirmer, nous allons transmettre la donnée : « OK, c'est confirmer ». Pour être sûr que la donnée, a bien été transmise, nous allons utiliser un « toast » pour afficher cette information :





### Modification dans le MainActivity.java

IMPORTANT : Remplacer **startActivity()** par l'appel d'une API **ActivityResultLauncher** :

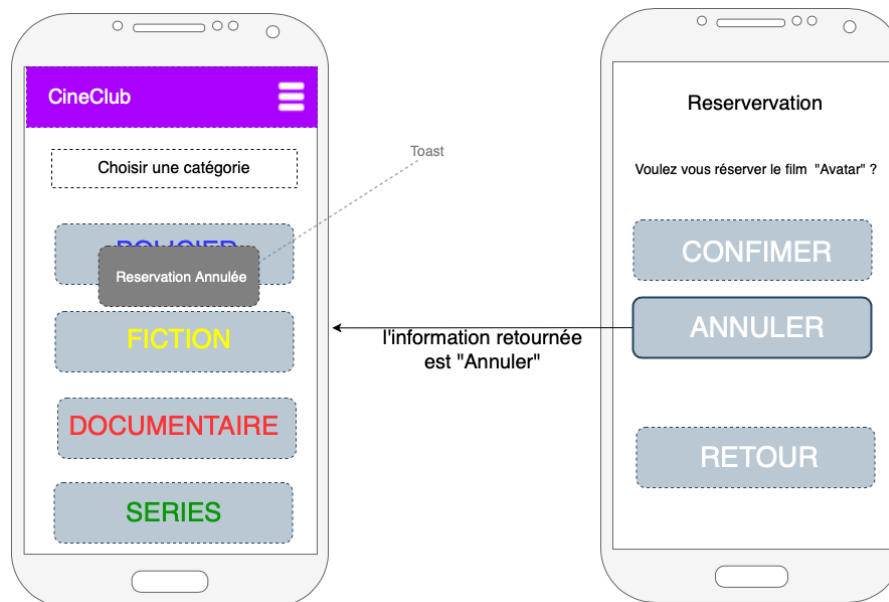
```
case R.id.menuReserver:
    //Log.i("LocDVD", "Menu:Réserver un film");
    //Lance l'activité qui affiche la fenêtre Réservation
    Intent intentReserver = new Intent(MainActivity.this, Reservation.class);
    reservationLauncher.launch(intentReserver);
    return true;
```

Il faut ensuite définir l'instance de l'API en spécifiant les actions à effectuer selon le code de retour reçu (resultCode)

```
// Traitement du résultat de l'activité réservation
ActivityResultLauncher<Intent> reservationLauncher = registerForActivityResult(
    new ActivityResultContracts.StartActivityForResult(),
    new ActivityResultCallback<ActivityResult>() {
        @Override
        public void onActivityResult(ActivityResult result) {
            if (result.getResultCode() == Activity.RESULT_OK) {
                Toast.makeText(MainActivity.this, "Réservation confirmée", Toast.LENGTH_SHORT).show();
            }
        }
    });
```

TESTEZ !!!

**Travail à faire : Finir de coder la partie manquante pour le bouton annuler**





## A RETENIR

### **requestCode**

Afin que l'activité émettrice puisse distinguer les différents résultats possibles, elle associe à chaque démarrage un numéro identifiant à la demande (requestCode). Dans notre exemple l'identifiant est 1.

### **onActivityResult(requestCode, data)**

Cette méthode permet de récupérer le code du résultat.

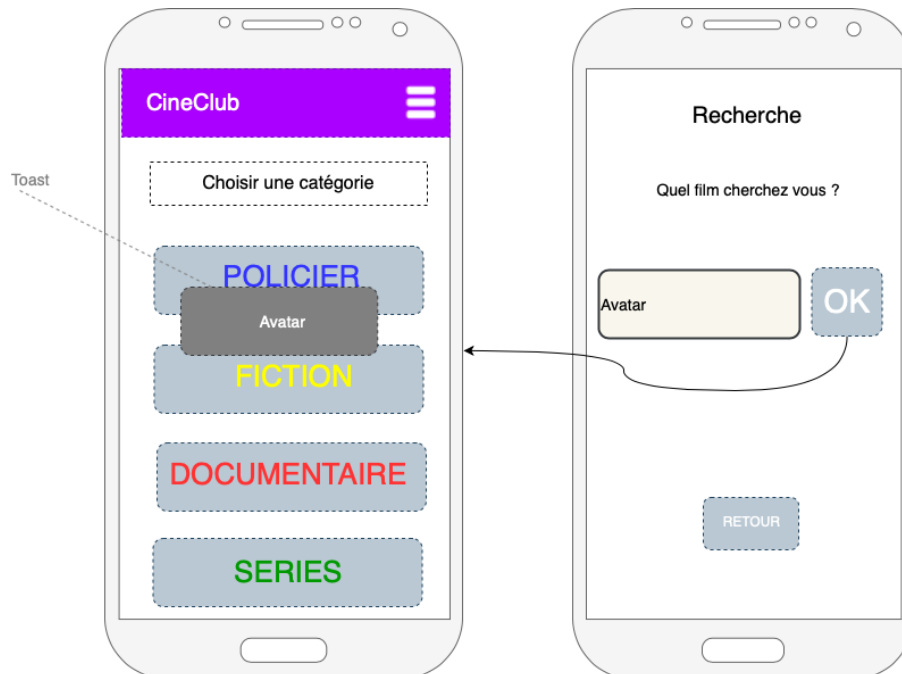
Ce code résultat est représenté par des constantes statiques comme Activity.RESULT\_OK ou Activity.RESULT\_CANCELED (voir activité Informations). Ces constantes sont implémentées dans la classe Activity.



### 3 Activité Recherche : Echange bidirectionnel

#### 3.1 Principe

Nous reprenons l'activité précédente mais au lieu d'envoyer un « **resultCode** » (qui a été prédéfini, donc limité) nous allons envoyer une donnée qui sera fournie par l'utilisateur.



#### 3.2 Ajouter un PlainText

Créer l'activité « **Recherche** ».

- **File / New / Activity / EmptyActivity**
- La création de l'activity : **Recherche** génère le layout : **activity\_recherche**

Dans le layout : **activity\_recherche**

- Ajouter un **PlainText** par la commande : **palette > Text > PlainText**
- Supprimez (vider) l'attribut « **text** » il ne sera pas utilisé ici.
- Ajoutez-lui un attribut **hint** qui vaut **@string/txtEditRecherche** pour obtenir le résultat suivant :

- Attribut id du **PlainText** : **edtRecherche**
- Pour le bouton recherche : **btnRecherche**



### 3.3 La classe Recherche.java

#### Réception de la donnée

Définir deux attributs privés : **btnCherche** et **edtCherche**, qui sont de type respectif : **Button** et **EditText**.

```
public class Recherche extends AppCompatActivity
{
    // Définition de 2 attributs privés
    private Button btnCherche;
    private EditText edtCherche;
```

Pour les associer à la vue, on utilisera la méthode **findViewById()**, dans le méthode **onCreate()**.

```
@Override
protected void onCreate(Bundle savedInstanceState)
{
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_recherche);
    // Association des attributs privés à la vue
    edtCherche = findViewById(R.id.edtRecherche);
    btnCherche = findViewById(R.id.btnRecherche);
```

Vous devez savoir utiliser la méthode **findViewById()** maintenant, si ce n'est pas le cas, retournez aux TP précédents.

Pour tester le fait que l'on reçoit bien la donnée, nous allons tester le code suivant, toujours dans la méthode **onCreate()** de la classe Recherche :

```
// Création du listener
btnCherche.setOnClickListener(new Button.OnClickListener()
{
    @Override
    public void onClick(View view)
    {
        Toast toast = Toast.makeText(getApplicationContext(), edtCherche.getText(), Toast.LENGTH_SHORT);
        toast.show();
    }
});
```

Il s'agit simplement, d'utiliser la méthode **getText()** de la classe **EditText** pour récupérer la donnée saisie.

**Remarque** : si une erreur apparaît sur le **OnClickListener**, cliquer droit et faire implémenter la méthode





### Envoyer la donnée depuis Recherche

L'intent est un pont entre les différentes activités. C'est donc à l'intent que nous attachons la ou les données, pour les transmettre à une autre activité. Cela se fait par la méthode suivante :

#### **putExtra ( key, value)**

C'est une **méthode non static**, de la classe **Intent**.

La méthode **putExtra** permet d'ajouter des données à transmettre.

Ces données doivent être structurées sous forme de **couples clé-valeur**.

Exemple ci-dessous : titre est la clé, et la valeur est le texte saisi dans le widget EditText

La méthode **setResult()** prend ici un paramètre supplémentaire, c'est l'intent qui contient notre donnée.

Remarque : il peut en contenir plus, il suffit d'appeler plusieurs fois la méthode **putExtra()** en créant des clés uniques.

Voici le code :

```
public void onClick(View view)
{
    Toast toast = Toast.makeText(getApplicationContext(), edtCherche.getText(), Toast.LENGTH_SHORT);
    toast.show();

    Intent intent = new Intent (Recherche.this, MainActivity.class);
    intent.putExtra("titre", edtCherche.getText().toString());
    setResult(Activity.RESULT_OK, intent);
    finish();
}
```

### Réception de la donnée dans MainActivity

Les données expédiées par l'activité enfant « **Recherche** » se trouvent dans data dans un certain **format (clé-valeur)**. Nous allons maintenant voir comment l'activité principale réceptionne ces données.

## **3.4 Modifier le lancement de l'activité**

Très important, il faut modifier le **startActivity()** et créer une instance *ActivityResultLauncher* spécifique qui va récupérer le titre recherché .

Voici le code :

```
case R.id.menuRechercher:
    //Log.i("LocDVD", "Menu:Rechercher un film");
    //Lance l'activité qui affiche la fenêtre Recherche
    Intent intentRecherche = new Intent(MainActivity.this, Recherche.class);
    rechercheLauncher.launch(intentRecherche);
    return true;
```



### 3.5 Ajouter l'instance d'API reservationLauncher

L'intent sera récupérer via la méthode `getData()` sur le paramètre `result` et on utilisera la méthode `getStringExtra()` sur l'intent pour récupérer la chaîne de caractères saisie lors de la recherche.

```
// Traitement du résultat de l'activité recherche
ActivityResultLauncher<Intent> rechercheLauncher = registerForActivityResult(
    new ActivityResultContracts.StartActivityForResult(),
    new ActivityResultCallback<ActivityResult>() {
        @Override
        public void onActivityResult(ActivityResult result) {
            if (result.getResultCode() == Activity.RESULT_OK) {
                Intent data = result.getData();
                String titre = data.getStringExtra("titre");
                Toast.makeText(MainActivity.this, titre, Toast.LENGTH_SHORT).show();
            }
        }
    });
```

## A RETENIR

### « L'intent est un PONT entre 2 activités »

Les données ne sont pas transmises directement. Elles sont transportées par un intent.

### EditText

EditText est la classe qui permet de manipuler les PlainText. `.getText()` permet de récupérer la valeur saisie

### intent.putExtra(key , value)

Les données sont associées à l'intent par la méthode `putExtra()`. Elles ont une structure de clé- valeur

### result.getData()

Ici, nous avons vu que la méthode `getData()` qui retourne un intent.

`Intent data = result.getData();`

Cette méthode permet de récupérer l'intent de l'activité source ainsi que les données envoyées

### intent.getStringExtra( key )

Permet de récupérer la valeur de la donnée associée à la clé

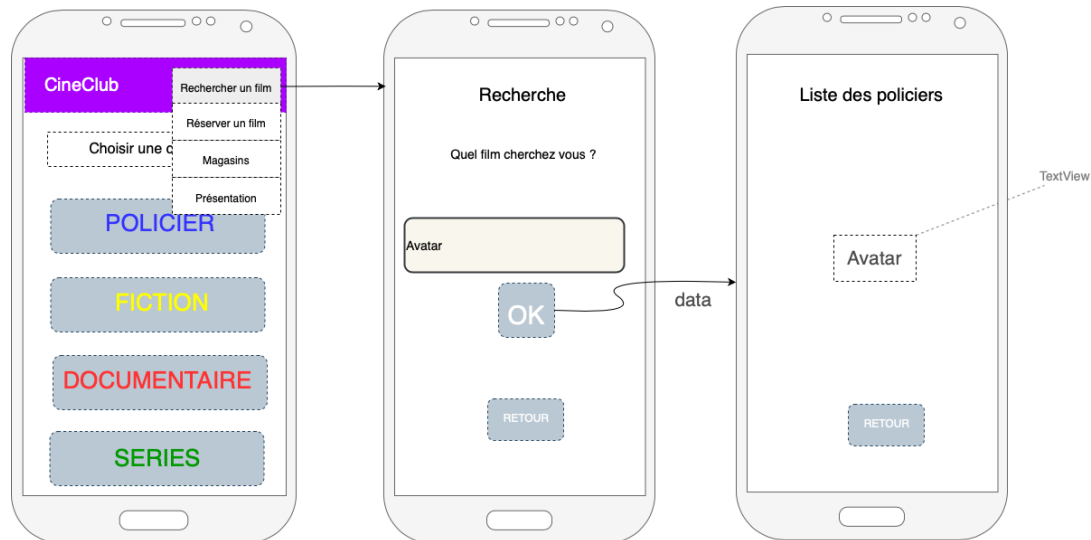


## 4 Activité Policier : Echange unidirectionnel

Pour cet exercice, nous reprendrons l'exercice précédent, donc vous devez avoir tout fait. Au lieu de revenir sur l'activité principale, vous allez envoyer la donnée sur une autre activité, par exemple l'activité **Policier**

### 4.1 Principe

Voici schématiquement, ce que vous devez faire :



### 4.2 Layout Activity\_Policier

- Ajoutez un **textView** possédant l'**id** : titreDemande
- Placer le au centre du layout
- Donner l'attribut **text** à « Titre demandé » en lui associant une ressource string.

### 4.3 Depuis l'activité Recherche

#### On change la destination

Vous ne devez plus retourner vers l'activité principale **MainActivity** mais vers l'activité **Policier**.

Pour cela, vous devez lui indiquer l'activité **Policier** comme nouvelle destination, en modifiant l'**intent** dans la méthode **onclick()** de la classe **Recherche**

```
// Création du listener
btnCherche.setOnClickListener(new Button.OnClickListener()
{
    @Override
    public void onClick(View view)
    {
        //Toast toast = Toast.makeText(getApplicationContext(), edtCherche.getText(), Toast.LENGTH_SHORT);
        //toast.show();

        Intent intent = new Intent (Recherche.this, Policier.class);
        intent.putExtra("titre", edtCherche.getText().toString());
        startActivity(intent);
    }
});
```



#### 4.4 Dans l'activité Policier

Vous ne passez pas par la méthode **onActivityResult()**

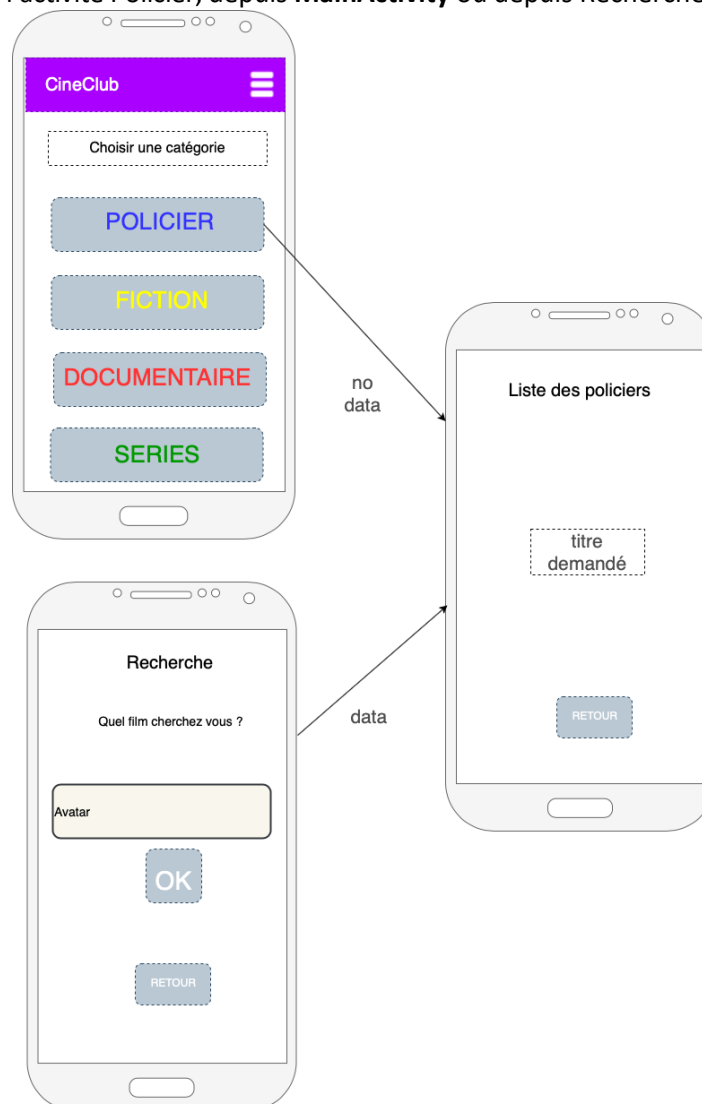
Vous devez récupérer votre « intent », directement depuis la création, c'est à dire que vous devez écrire votre code dans la méthode **onCreate()**

Vous avez accès à votre **intent**, par la méthode accesseur **getIntent()**

```
protected void onCreate(Bundle savedInstanceState) {  
    super.onCreate(savedInstanceState);  
    setContentView(R.layout.activity_policier);  
  
    //Venant de recherche  
    String titre = this.getIntent().getStringExtra("titre");  
    TextView titreDemande = findViewById(R.id.titreDemande);  
    titreDemande.setText(titre);  
}
```

#### Remarque

Maintenant, on peut arriver à l'activité Policier, depuis **MainActivity** ou depuis Recherche





## 4.5 Tout type de données

| Type            | Transmettre                     | Récupérer                       | Exemple     |
|-----------------|---------------------------------|---------------------------------|-------------|
| String          | putStringExtra(key, value)      | getStringExtra(key, value)      | user_name   |
| Int             | putIntExtra(key, value)         | getIntExtra(key, value)         | user_id     |
| Float           | putFloatExtra(key, value)       | getFloatExtra(key, value)       | user_rating |
| Array of string | putStringArrayExtra(key, value) | getStringArrayExtra(key, value) | list_users  |

Vous pouvez être amené à transmettre tout type de données :

Et ainsi de suite, vous avez compris la logique, je vous invite à regarder la documentation au besoin.

Pour les curieux : Utilisation de Bundle pour transmettre des données, veuillez suivre le lien ci-dessous  
<https://medium.com/@haxzie/using-intents-and-extras-to-pass-data-between-activities-android-beginners-guide-565239407ba0>